marrow

The embeddable agent runtime — and the reference implementation of ARI, the Agent Runtime Interface

The runtime layer for AI agents that run **where you can't ship Python**: robots, edge devices, and native applications.

Presented by **[Your Name]**, Founder — `[email]` · `github.com/bencrooks-dev/marrow`

pre-alpha · 2026 · replace bracketed fields before sending

The shift is real — and moving to production

- **33%** of enterprise software apps will include agentic AI by **2028**, up from **<1%** in 2024 — *Gartner, 2025*
- **57%** of teams already run AI agents **in production** (n=1,340) — *LangChain State of AI Agents, 2025*
- **84%** of developers use or plan to use AI tools; **51%** use them daily — *Stack Overflow Developer Survey, 2025*

Agents are no longer demos. They're shipping. **The question is where they run.**

The problem: agents are escaping the data center

Agents are moving into places a Python server can't go:

- **Robots & drones** — hard real-time, no GC pauses
- **Edge / on-device** — constrained, often offline, local models
- **Native apps** — game engines, audio/DSP, trading systems, CAD

Every mature agent framework — **LangChain, LangGraph, CrewAI, AutoGen** — is **Python-locked by design.** None can be embedded in these targets without shipping a Python runtime.

Why "Python-locked" is fatal here — not just slower

- **Latency is the #2 production blocker (20%)** for agents today — *LangChain, 2025*
- Real-time control loops (100–1000 Hz) need **C/C++ for deterministic deadlines**; the GIL and interpreter overhead make Python the wrong layer — *robotics engineering consensus, 2026*
- You cannot ship a CPython runtime into a motor controller, a drone, or a customer's native binary.

This isn't a performance gap. It's a structural wall the incumbents can't cross.

The insight: own the layer they can't enter

ARI-Embedded — a runtime profile defined by constraints a Python framework *cannot* meet:

- native core, **no managed-language runtime required**
- **bounded memory**, no hot-path allocation
- mandatory **cancellation + wall-clock timeouts**
- near-zero dependency footprint

> A Python-locked framework cannot satisfy this without ceasing to be Python-locked. **That's the moat.**

What we built

marrow — a native **C++17** agent runtime (state · provider · tools · routing · lifecycle), embeddable with zero managed runtime, ergonomic from Python.

It is the **reference implementation of ARI** — an open standard for the agent runtime layer.

```
A2A   — agent-to-agent (across hosts)
MCP   — tool / context exchange          ← 97M+ SDK downloads/mo, now Linux Foundation
ARI   — the runtime that runs a turn     ← marrow implements this
Model provider (OpenAI / Anthropic / local)
```

ARI sits beneath MCP/A2A — complementary, not competitive. Their adoption is our tailwind.

The play: don't just build a product — set the standard

The durable move (OpenStack, Open Compute, POSIX): **own the standard AND its reference implementation.**

1. **The spec** — ARI, a language-neutral contract (shipped, v0.1)
2. **The reference implementation** — marrow (shipped)
3. **The conformance kit** — makes "ARI-conformant" *falsifiable* by anyone (shipped)
4. **A security baseline** — STRIDE threat model + hardening + CI scanning (shipped)

Own the layer *and* the language the industry uses to describe it.

Market

Framed as ranges where sources are syndicated; analyst-grade anchors called out.

Market	Size (2025)	Growth
AI agents / agentic AI	~$7.0–7.8B	~44–50% CAGR
Edge AI	~$12–26B	~18–37% CAGR
Humanoid robotics (TAM)	—	**~$38B by 2035** *(Goldman Sachs)*
On-device / TinyML	~$1.5B	~20–38% CAGR

Bolder framing: Morgan Stanley models a **~$5T humanoid market by 2050** *(different scope & horizon — don't compare directly to Goldman's $38B)*.

Our wedge (embedded/embodied) is the fastest-growing enabling layer, not the biggest TAM today — that's the early-mover opportunity.

Why we win — the moat

- **Structural** — incumbents are Python; they can't follow into embedded/real-time
- **Standard ownership** — reference implementation of ARI + the conformance kit
- **Dogfood credibility** — Tilo (humanoid) as the lighthouse deployment *(status: integration in progress — replace with current state)*
- **Ecosystem-complementary** — rides MCP/A2A adoption instead of fighting it
- **Security-first** — threat model + hardening shipped before 1.0 (table stakes for infrastructure others depend on)

Traction & status (honest)

Shipped:

- ARI spec v0.1 + reference implementation (marrow, 0.1.0)
- Conformance kit (falsifiable conformance)
- STRIDE threat model + security hardening + CI security scanning

Stage: pre-alpha. External users: 0. First deployment target: Tilo.

Replace the italic examples below with real numbers — do not present as-is:

- Design partners: *e.g., 1 robotics + 1 trading (illustrative)*
- GitHub / downloads / community: *e.g., [stars], [PyPI installs] (illustrative)*
- Pilots / LOIs: *e.g., none yet, or list (illustrative)*

Roadmap

1. **Dogfood** — ARI-Embedded build running in Tilo (real robot = real proof)
2. **Second proof point** — embed in a latency-sensitive native stack
3. **Second implementation** — a runtime in another language passes the kit (→ "standard" becomes literal)
4. **Adapters** — expose ARI tools over MCP; A2A interop
5. **Neutral governance** — move the spec to an open foundation as adoption justifies

Business model

Recommended: open core *(refine before presenting):*

- **Open core** — Apache-2.0 runtime + spec; commercial add-ons (managed fleet, observability, certified embedded builds, support/SLA)
- **Conformance / certification** program for vendors
- **Enterprise support** for embedded/robotics deployments

Apache-2.0 explicitly permits embedding in proprietary products — adoption-friendly by design.

The ask

Raising: $[1.5M] pre-seed *(illustrative — set your real number)*

- **Use of funds:** ~50% engineering (embedded runtime + a second-language implementation), ~25% Tilo / robotics integration, ~15% standards evangelism + design-partner support, ~10% ops
- **Milestones this funds (12–18 mo):** ARI-Embedded running in Tilo in production · first *external* conformant implementation · `[N]` design partners · conformance CI gate green

All figures illustrative — replace with your real plan before presenting.

Team

[Your Name] — Founder. *[Background: AI engineering + systems/C++ + robotics — replace with your real bio]*

Advisors: *[names / roles — replace]*

Why this team: the embedded-systems + AI + standards-creation combination this requires — fill with your real edge.

Why now

- Agents are hitting production **and** hitting the latency wall (LangChain, 2025)
- Embodied/edge AI is inflecting (Goldman/Morgan Stanley humanoid forecasts)
- The runtime layer is **unclaimed** — MCP took tools, A2A took messaging, **no one owns the embeddable runtime**
- Standards get set early. **The window is open now.**

marrow — the runtime you embed where Python can't go.

`[SUPPLY THIS: contact / next step]`

Sources

All figures trace to `docs/market-research.md` (each with source + year + URL).

- **Gartner, 2025** — 33% of enterprise apps include agentic AI by 2028 (from <1% in 2024)
- **LangChain State of AI Agents, 2025** (n=1,340) — 57% run agents in production; latency = #2 blocker (20%)
- **Stack Overflow Developer Survey, 2025** — 84% use/plan AI; 51% daily
- **Goldman Sachs, 2024/25** — humanoid TAM ~$38B by 2035 · **Morgan Stanley, 2025** — ~$5T by 2050
- **Anthropic, Dec 2025 (reported)** — MCP 97M+ monthly SDK downloads, 10,000+ active servers; donated to Linux Foundation
- AI-agents / Edge-AI / TinyML TAM ranges — Grand View, MarketsandMarkets, Precedence, DataM (syndicated; ranges)